Quantum Network-Based Prediction of Cancer Driver Genes

Patrícia Marques,[1] Andreas Wichert,[1,2] and Bruno Coutinho[3,4]

[1] *Instituto Superior Técnico, University of Lisbon, Lisboa, Portugal*
[2] *GAIPS, INESC-ID, Porto Salvo, Portugal*
[3] *Instituto de Telecomunicações, Lisboa, Portugal*
[4] *German Aerospace Center (DLR), Cologne, Germany*

(Dated: August 7, 2025)

Identification of cancer driver genes is fundamental for the development of targeted therapeutic interventions. The integration of mutational profiles with PPI networks offers a promising avenue for the detection of such driver genes [1, 2]. However, the application of quantum computing methodologies in this domain remains largely unexplored. In this study, we introduce a supervised quantum framework for cancer driver gene prediction that combines mutation scores with network topology via a novel state preparation scheme, *Quantum Multi-order Moment Embedding* (QMME), motivated by the classical method of Gumpinger et al [3]. QMME encodes low-order statistical moments over the mutation scores of a node's immediate and second-order neighbors, and encodes this information into quantum states. These are used as inputs to a kernel-based quantum binary classifier that discriminates known driver genes from others. Through a simulation on a real PPI network, we confirm that this approach achieves competitive predictive performance relative to classical baselines. In a brief complexity analysis, we identify the potential of our approach in uncovering a quantum speedup for quantum network-based prediction of cancer driver genes. This work underscores the potential of supervised quantum graph learning frameworks to advance biological discovery.

I. INTRODUCTION

A central objective in oncology is to identify and catalogue genes underlying human cancer [4–6]. While genes with high mutation rates are likely drivers, most *cancer driver genes* have few mutations. In these cases, frequency-based methods alone cannot reliably distinguish rare cancer driver genes from irrelevant ones [5]

One explanation lies in the networked nature of cellular processes: genes operate within pathways and complexes, so cancer may result from disruption at the pathway level rather than mutations in specific individual genes. Recent research has embraced this interaction-based perspective of cancer, integrating biological networks with statistical measures of gene-cancer associations to identify novel candidate cancer driver genes [1, 7–9]. In these networks, nodes correspond to genes and edges represent relationships between them. Protein-protein interaction (PPI) networks [10–12] are a widely used representation of gene interactions, as they integrate information from diverse data sources, tissues, and molecular processes.

Previous studies have combined PPI networks with gene-level cancer association scores, using clustering or network propagation methods. A common choice for these gene scores is the MutSig P-value [13, 14], which estimates the significance of observed mutations by considering (i) the overall mutational burden, (ii) clustering of mutations within a gene's coding sequence, and (iii) the functional impact of mutations. NetSig, for instance, focuses on the local neighborhood of each gene to identify cancer driver genes while correcting for node degree bias [1]. A few studies have explored supervised approaches, incorporating existing knowledge of known driver genes during prediction. Gumpinger et al. used Cancer Gene Census (CGC) annotations to train classifiers for discovering new cancer driver genes [3]. This method models the task as a node-level supervised classification problem in a molecular interaction network, combining network topology with mutation-score features. It achieved more than a 10% improvement in area under the precision–recall curve (AUPRC) over baseline methods.

Network-based approaches derive their strength from integrating weak mutation signals across interconnected genes rather than relying on strong evidence from individual genes. This demonstrates the value of graph-structured representations as a paradigm for cancer driver gene discovery. However, as biological networks continue to grow in size and complexity, classical methods face increasing challenges in scalability and representational efficiency. This motivates the exploration of quantum computing as an alternative framework for representing and processing such structured data, including PPI networks. In particular, quantum machine learning (QML), has been proposed as a tool with potential for inference tasks [15–18]. Recent work has begun applying quantum computing to biological network inference problems, including link prediction tasks [19–21], demonstrating potential speedups over classical baselines, further supporting the potential of quantum approaches in this domain.

Classical methods often rely on local statistical descriptors, such as neighborhood means or low-order moments, to capture structural information in graphs [3, 22]. However, quantum embedding strategies that incorporate such statistics remain unexplored. Moreover, many existing QML approaches neglect the gate complexity of state preparation, limiting their practicality as end-to-end quantum solutions [23, 24]. Motivated by these gaps, we

We need an illustrative figure here

introduce a quantum protocol that embeds low-order statistical moments of features in the one-hop and two-hop neighborhoods into amplitude-encoded quantum states.

In this work, we introduce the *Quantum Multi-order Moment Embedding* (QMME) protocol, which encodes local graph statistics into quantum states for inference. We apply this feature map to a PPI network, where nodes represent genes and features correspond to MutSig P-values. Thus, we design an almost fully quantum pipeline for node-level binary classification of genes as cancer drivers. Classification is performed using the swap-test classifier [25], a non-parametric, distance-based method that employs a quantum kernel based on state similarity.

The entire process, from embeddings to inference, is carried out within the quantum domain, requiring no classical optimization. It relies only on minimal data access assumptions, enabling straightforward circuit design, and involves only limited classical post-processing, inherent to the swap-test classifier. The pipeline explicitly addresses state preparation with practical circuit construction, ensuring embeddings can be constructed with logarithmic qubit resources and polynomial-depth circuits. We validate our approach through numerical simulations on real graphs, evaluating classification performance alongside quantum resource metrics such as circuit depth and query complexity. This work establishes a proof-of-principle for end-to-end quantum learning on graph-structured biological data and lays groundwork for future quantum models in complex network inference tasks.

II. PROBLEM SETUP

Formally let's represent it as

Let $\mathcal{G} = (V, E, \boldsymbol{x})$ be an unweighted, undirected graph with $|V| = N$ nodes, where $V = [N]$ denotes the set of node indices and $E \subseteq V \times V$ is the edge set. The connectivity of $\mathcal{G}$ is described by its adjacency matrix $A \in \{0,1\}^{N\times N}$, where $A_{vu} = 1$ indicates the presence of an edge between nodes v and u, and $A_{vu} = 0$ its absence. Define the immediate neighborhood of v as

$$\mathcal{N}_v^1 = \{u \in V \mid A_{vu} = 1\}. \tag{1}$$

We assume $\mathcal{N}_v^1$ is ordered by increasing node index. This allows us to define the function $r(v, \ell)$ as

$$r(v,\ell) := \mathcal{N}_v^1(\ell), \quad \ell \in \{0, \ldots, |\mathcal{N}_v^1| - 1\}, \tag{2}$$

which returns the index of the ℓ-th neighbor of v in $\mathcal{N}_v^1$. Equivalently, $r(v, \ell)$ is the column index of the ℓ-th nonzero entry in row v of A. If $\ell \geq |\mathcal{N}_v^1|$, i.e., the function returns a flag value indicating no such neighbor exists.

The two-hop neighborhood $\mathcal{N}_v^2 \subseteq V$ is defined as

$$\mathcal{N}_v^2 = \{w \in V \mid \exists u \in V,\, A_{vu} = 1 \wedge A_{uw} = 1\}. \tag{3}$$

Each node $v \in V$ has a scalar feature value $x_v \in \mathbb{R}$. The collection of all node features is denoted by the vector

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_N) \in \mathbb{R}^N. \tag{4}$$

A PPI network can be represented as a graph, where nodes are genes and edges indicate interactions between genes. Each gene's association with cancer can be quantified using the MutSig P-value [13, 14], which reflects statistical significance of mutations based on mutational burden, mutations clustering, and functional impact [3].

B. Node Embeddings

The low-order origin (or raw) moments of a scalar random variable $X \sim \mathcal{P}$ are defined as $\mathbb{E}[X^i]$, where i denotes the order of the moment [26]. These moments are uncentered (i.e., calculated about the origin) and capture fundamental properties of the distribution. These include the mean $\mathbb{E}[X]$, the uncentered quadratic $\mathbb{E}[X^2]$, cubic $\mathbb{E}[X^3]$, and quartic $\mathbb{E}[X^4]$ moments. These origin moments serve as the building blocks for the more familiar summary statistics of variance, skewness, and kurtosis, which are centered or standardized variants derived from them.

Formally, we define the *origin moment embedding* as the mapping

$$\overline{\varphi}(X) = \big[\mathbb{E}[X],\ \mathbb{E}[X^2],\ \mathbb{E}[X^3],\ \mathbb{E}[X^4]\big], \tag{5}$$

which collects the first four origin moments of X.

Since the true distribution $\mathcal{P}$ is generally unknown, these moments are typically estimated empirically. For a finite sample $\boldsymbol{s} = [s_1, \ldots, s_k]$ of X, with $s_j \in \mathbb{R}$, the corresponding *sample raw moment embedding* is given by

$$\varphi(\boldsymbol{s}) = \left[\sum_{j=1}^{k} \frac{s_j}{k},\ \sum_{j=1}^{k} \frac{s_j^2}{k},\ \sum_{j=1}^{k} \frac{s_j^3}{k},\ \sum_{j=1}^{k} \frac{s_j^4}{k}\right]. \tag{6}$$

which returns the first four sample origin moments from the sample, thus, describing its distributional shape.

The hop order $c \in \{1, 2\}$ specifies the neighborhood distance from node v. Given $\boldsymbol{x}$ defined in eq. (4),

$$\mathcal{X}_v^c = \{x_u \in \boldsymbol{x} \mid u \in \mathcal{N}_v^c\}, \tag{7}$$

denotes the set of feature values of nodes at hop-distance c from v, which are samples from a local distribution $\mathcal{P}_v^c$. The function $\varphi(\cdot)$ is applied to each $\mathcal{X}_v^c$, such that

$$\boldsymbol{m}_v^c := \varphi(\mathcal{X}_v^c) \in \mathbb{R}^4. \tag{8}$$

For vertex $v \in V$, the node embedding is then the concatenation of $\boldsymbol{m}_v^c$ from immediate and two-hop neighborhoods:

$$\boldsymbol{m}_v = \big[\boldsymbol{m}_v^1, \boldsymbol{m}_v^2\big] \in \mathbb{R}^8, \tag{9}$$

defining the *node embedding function* $\boldsymbol{m}_v : V \to \mathbb{R}^8$.

WORD REPETITION

1- Acho que nos dava jeito uma imagem nas primeiras paginas. So ilustrativa a esplicar o problem.

2_ A secao da complexity analysis precisa de trabalho. Temos de calcular para matrizes esparsas e nao esparsas. E discutir isso um pouco. Ve o segundo artigo do JOao para referencia.

3- Tens muitos paragrafos ainda. Muda de paragrafo apenas se tiveres a mudar de assunto.

Figure titles are usually in bold.

If a figure is composed of smaller figure, each smller figure needs to be label (a) (b), with (a) and (b) mentioned in the caption

C. Problem Statement

We consider a supervised binary classification task on a graph $\mathcal{G} = (V, E, \boldsymbol{x})$, where a subset of nodes $V_l \subseteq V$ is labeled. For these nodes, $y_v = 1$ for $v \in V_1 \subset V_l$ and $y_v = 0$ for $v \in V_0 \subset V_l$, corresponding to the positive and negative classes, respectively. This is a node-level classification task, i.e., the aim is to predict labels for individual nodes, as opposed to edge-level tasks such as link prediction. Our goal is to predict the label $\hat{y}_v$ for any node $v \in V$, using a method that combines a *quantum node embedding* unitary operator $U_{\mathcal{G}}$, derived from the node features $\boldsymbol{x}$ and the connectivity E of $\mathcal{G}$, with a quantum binary classifier $\mathcal{C}$. This combination is used to predict whether a node v belongs to the positive or negative class. That is,

$$\mathcal{C}\left(U_{\mathcal{G}} |v\rangle |0\rangle^{\otimes \mathfrak{m}}\right) = \hat{y}_v, \tag{10}$$

where $\hat{y}_v$ denotes the predicted class label for node v, and $\mathfrak{m}$ is the number of data qubits.

The objective is to explicitly define $U_{\mathcal{G}}$ as the state preparation step and design the full quantum circuit for the supervised classification of cancer driver genes. The input is a PPI network where the node features $\boldsymbol{x}$ are per-gene MutSig P-values. For the labeled data, a set of positively labeled genes V_1 is obtained from CGC data, so the positive class corresponds to cancer driver genes.

III. QUANTUM MULTI-ORDER MOMENT EMBEDDING (QMME)

A. Input Model

We base our algorithm on having query access to the graph $\mathcal{G} = (V, E, \boldsymbol{x})$ through the adjacency list model for bounded-degree graphs [27–29]. In this input model, the graph has N vertices and bounded degree $s = 2^{\mathfrak{s}}$, i.e., at most s nonzero entries in each row of the adjacency matrix [30, 31]. To accommodate an attributed network, we extend this model by adding a feature query. Thus, the three available query operations are the following:

1. **neighbor query:** given $v \in V$ and $\ell \in \mathbb{Z}$, returns the ℓ-th neighbour of v if $\ell \leq k_v$, and $*$ otherwise,
2. **feature query:** given $v \in V$, returns the value x_v,
3. **degree query:** given $v \in V$, returns the degree k_v.

A quantum equivalent of this input model can be described by defining three corresponding unitary operators O_L, $\tilde{O}_X$, and $\tilde{O}_K$, acting on appropriate Hilbert spaces. Here, O_L provides access to a node's immediate neighborhood, $\tilde{O}_X$ encodes node features, and $\tilde{O}_K$ allows access to the number of c-hop neighbors. The so-called *oracles* act as:

$$O_L : |v\rangle |\ell\rangle \mapsto |v\rangle |r(v, \ell)\rangle , \tag{11}$$

$$\tilde{O}_X : |v\rangle |0^{d'}\rangle \mapsto |v\rangle |\tilde{x}_v\rangle , \tag{12}$$

$$\tilde{O}_K : |v, c\rangle |0^{d'}\rangle \mapsto |v, c\rangle |\tilde{k}^c\rangle \tag{13}$$

Although oracle implementation in real hardare is a topic of active ressearch, our work assumes

ndex, $\tilde{x}_v$ the feature value of the node neighbors.

be challenging [32], our work assumes coherent access, allowing queries to be made in superposition, which is a standard assumption in the theoretical framework of quantum algorithms. We also assume access to the controlled versions of the oracles O_L, $\tilde{O}_X$, and $\tilde{O}_K$ and their inverses. Since these oracles are unitary and represented by real-valued matrices, their inverses reduce to their transposes.

We assume that neighbor and feature queries, whether classical or quantum, each count as $\mathcal{O}(1)$ in query complexity. The degree query can be emulated by $\mathcal{O}(\log s)$ neighbor queries via binary search [28]. We introduce $\tilde{O}_K$ as a generalization of the degree query, enabling access to the size of a c-hop neighborhood, with $c \in \{1, 2\}$. Thus, $\tilde{O}_K$ can be constructed from O_L with $\mathcal{O}\left((\log s)^2\right)$ queries.

TO MAKE WORD REPETITION

B. Rotation Operators

The goal of the embedding circuit is to encode local graph statistics into the amplitudes of quantum states. This is done using controlled single-qubit rotations, where the rotation angles are determined by values encoded in the computational basis and retrieved from the oracles. To this end, we define two rotation operators, F_X and $F_{K^{-1}}$.

Given a node feature $x_v \in \mathbb{R}$ with $|x_v| \leq 1$, and its binary encoding $\tilde{x}_v$, the *feature rotation operator* F_X [30] is defined as

$$F_X : |0\rangle |\tilde{x}_v\rangle \mapsto \left(x_v |0\rangle + \sqrt{1 - |x_v|^2}\, |1\rangle\right) |\tilde{x}_v\rangle , \tag{14}$$

which corresponds to a single-qubit rotation $R_y(\theta)$ with angle $\theta = 2 \arcsin(x_v)$. This encodes the scalar value x_v into the amplitude of the $|0\rangle$ component. Similarly, we define the *degree-inverse rotation operator* $F_{K^{-1}}$ [33] as

$$F_{K^{-1}} : |0\rangle |\tilde{k}_v^c\rangle \mapsto \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left|\frac{1}{k_v^c}\right|^2}\, |1\rangle\right) |\tilde{k}_v^c\rangle . \tag{15}$$

These operators are implemented using multi-controlled R_y rotation gates [30], where the control register holds the binary input $\tilde{x}_v$ or $\tilde{k}_v^c$. The rotation angles are $\theta(\tilde{x}_v) = 2 \arcsin(x_v)$ and $\theta(\tilde{k}_v^c) = 2 \arcsin(1/k_v^c)$, respectively.

C. Feature and Degree-Inverse Oracles

To amplitude-encode the node embedding $\boldsymbol{m}_v$ into quantum states, we introduce two derived oracles O_X and O_K^{-1}, which use the basis-encoding input oracles $\tilde{O}_X$ and $\tilde{O}_{K^{-1}}$, along with rotation operators F_X and F_K^{-1}. The *feature oracle* O_X encodes powers of the feature value x_v conditioned on the control register $|i\rangle$. Specifically, the amplitude of the component where the ancilla register is in state $|0\rangle^{\otimes 4}$ is mapped to x_v^i, which relates to the i-th order term in the raw moment embedding function $\varphi(\cdot)$ from eq. (6). The oracle O_X acts on a control register, where the integer i in the control register $|i\rangle$ dictates how many of the ancilla qubits are rotated by F_X, as follows:

$$\begin{aligned}
&|v,i\rangle\,|0\rangle^{\otimes(4+d')} \xrightarrow{\tilde{O}_X} |v,i\rangle\,|0\rangle\,|\tilde{x}_v\rangle \\
&\quad\xrightarrow{C\text{-}F_X} |v,i\rangle\Big(x_v\,|0\rangle+\sqrt{1-|x_v|^2}\,|1\rangle\Big)^{\otimes i}|0\rangle^{\otimes(4-i)}\,|\tilde{x}_v\rangle \\
&\quad\xrightarrow{\tilde{O}_X^T} |v,i\rangle\left(x_v\,|0\rangle+\sqrt{1-|x_v|^2}\,|1\rangle\right)^{\otimes i}|0\rangle^{\otimes(4-i+d')}\,.
\end{aligned} \tag{16}$$

Omitting the intermediate register $|\tilde{x}_v\rangle$, fig. 1 summarizes the circuit of oracle O_X, which can be expressed as

$$O_X : |v,i\rangle|0\rangle^{\otimes 4} \mapsto |v,i\rangle\Big(x_v\,|0\rangle+\sqrt{1-|x_v|^2}\,|1\rangle\Big)^{\otimes i}|0\rangle^{\otimes(4-i)}\,. \tag{17}$$

The degree-inverse oracle $O_{K^{-1}}$ amplitude-encodes the inverse degree scaling factors $1/k_v^c$, required to emulate the average present in the moment embedding function $\varphi(\cdot)$, included in the node embedding $\boldsymbol{m}_v$. It performs a controlled rotation that encodes the value $1/k_v^c$ as the amplitude of the $|0\rangle$ state of an ancilla qubit, as follows:

$$\begin{aligned}
&|v,c\rangle\,|0\rangle^{\otimes(1+d')} \xrightarrow{\tilde{O}_K} |v,c\rangle\,|0\rangle\,|\tilde{k}_v\rangle \\
&\quad\xrightarrow{F_{K^{-1}}} |v,c\rangle\left(\frac{1}{k_v^c}\,|0\rangle+\sqrt{1-\left|\frac{1}{k_v^c}\right|^2}\,|1\rangle\right)|\tilde{k}_v\rangle \\
&\quad\xrightarrow{\tilde{O}_K^T} |v,c\rangle\left(\frac{1}{k_v^c}\,|0\rangle+\sqrt{1-\left|\frac{1}{k_v^c}\right|^2}\,|1\rangle\right)|0\rangle^{\otimes d'}\,.
\end{aligned} \tag{18}$$

As shown in fig. 2, the intermediate register $|\tilde{k}_v\rangle$ is omitted, and the oracle is represented as:

$$O_{K^{-1}} : |v,c\rangle\,|0\rangle \mapsto |v,c\rangle\left(\frac{1}{k_v^c}\,|0\rangle+\sqrt{1-\left|\frac{1}{k_v^c}\right|^2}\,|1\rangle\right). \tag{19}$$

Together with the neighbor oracle O_L and Hadamard gates, the oracles O_X and $O_{K^{-1}}$ form the core building blocks of the QMME circuit, as depicted in Figure 2.

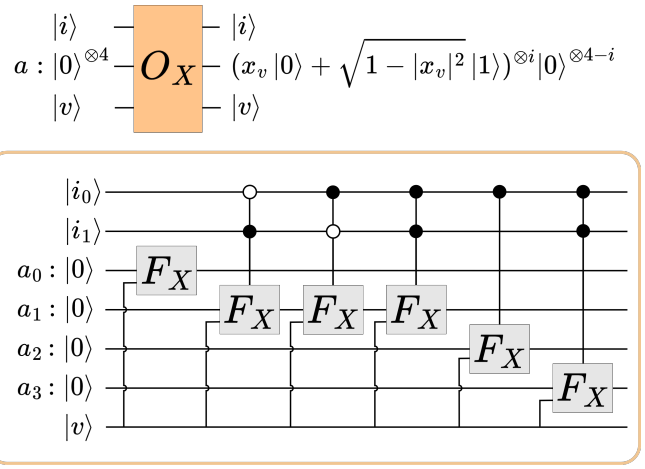


FIG. 1. Circuit of the feature oracle O_X, based on controlled versions of the rotation operator F_X. A two-qubit control register $|i\rangle$ selects how many of the four ancilla qubits undergo the rotation F_X: for $i = 00$, only the first qubit; for $i = 01$, the first two; for $i = 10$, the first three; and for $i = 11$, all four. A d'-qubit register is used internally but omitted for clarity.

D. QMME Algorithm

We define the QMME feature map as a unitary operator $U_{\mathcal{G}}$, which encodes node v into a quantum state via

$$U_{\mathcal{G}} : |v\rangle\,|0\rangle^{\otimes \mathfrak{m}} \mapsto |v\rangle\,|\boldsymbol{\psi}_v\rangle\,, \tag{20}$$

and refer to this mapping as the *quantum node embedding*. The second register (of $\mathfrak{m}$ qubits) is referred to as the *data register*. The total number of qubits is $\mathfrak{m} = 2n + 8$ (excluding work registers), and generally scales as $\mathcal{O}(n)$. A schematic of the circuit for $U_{\mathcal{G}}$ is shown in fig. 2. The resulting state $|\boldsymbol{\psi}_v\rangle$ consists of two components:

$$|\boldsymbol{\psi}_v\rangle = |\boldsymbol{\psi}_m\rangle + |\boldsymbol{\psi}_\perp\rangle\,, \tag{21}$$

where $|\boldsymbol{\psi}_m\rangle$ is the *neighborhood-moment component*, capturing the quantum analogue of the classical embedding $\boldsymbol{m}_v \in \mathbb{R}^8$, and $|\boldsymbol{\psi}_\perp\rangle$ is an orthogonal component.

Resulting from the QMME circuit, $|\boldsymbol{\psi}_m\rangle$ is defined as

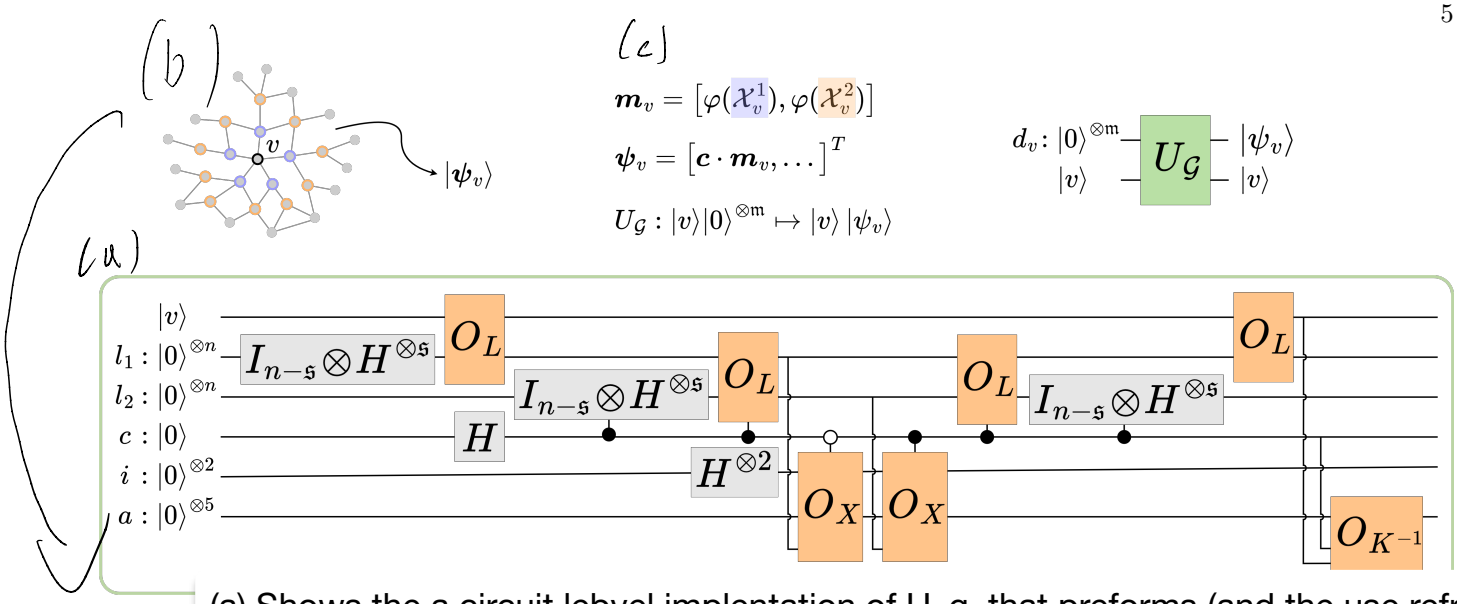
$$|\boldsymbol{\psi}_m\rangle = |\boldsymbol{c}\odot\boldsymbol{m}_v\rangle\,|0\rangle^{\otimes(\mathfrak{m}-3)} \tag{22}$$

$$= \sum_{c\in[2]} \frac{1}{2\sqrt{2}\,s^c} \sum_{i\in[4]} \varphi_i(\mathcal{X}_v^c)\,|c,i\rangle\,|0\rangle^{\otimes(\mathfrak{m}-3)} \tag{23}$$

$$= \sum_{j=0}^{7} c_j \cdot m_{v,j}\,|j\rangle\,|0\rangle^{\otimes(\mathfrak{m}-3)}\,, \tag{24}$$

where $\odot$ denotes the element-wise (Hadamard) product, $\varphi_i(\mathcal{X}_v^c)$ are the ith-order moment of the neighborhood $\mathcal{X}_v^c$, and the constant vector $\boldsymbol{c} \in \mathbb{R}^8$ is

$$\boldsymbol{c} = \frac{1}{2\sqrt{2}}\left[\frac{1}{s},\frac{1}{s},\frac{1}{s},\frac{1}{s},\frac{1}{s^2},\frac{1}{s^2},\frac{1}{s^2},\frac{1}{s^2}\right]. \tag{25}$$



(a) Shows the a circuit lebvel implentation of U_g, that preforms (and the use refrences to (b) and (c) to explain it)

FIG. 2. (encoding, with $\mathfrak{m} = \mathcal{O}(n)$. The output quantum state $|\psi_v\rangle \in \mathbb{R}^{2^{\mathfrak{m}}}$ captures neighborhood statistics of the graph $\mathcal{G} = (V, E, \boldsymbol{x})$ for node v, based on the raw moment embedding function $\varphi(\cdot)$ defined in eq. (9). Input data include synthetic graphs and real-world PPI networks with scalar node features such as mutation scores. The $\mathfrak{m}$-data register includes the neighbors registers (l_1) and l_2, the c qubit to indicate immediate or second-neighbors, the i two-qubit register corresponding to the index of the i-th moment, and the five-qubit ancilla register a.

The orthogonal part of the state is:

$$|\psi_\perp\rangle = \sum_{a \neq 0} |\psi_{\perp,a}\rangle\,|a\rangle\,, \tag{26}$$

where $|a\rangle$ refers to the computational basis states in the $(\mathfrak{m}-3)$ qubits, and $|\psi_{\perp,a}\rangle \in \mathbb{R}^{2^{(\mathfrak{m}-3)}}$

Let us call the base case where $i = 0$ and $c = 0$, and we aim to retrieve $m_{v,1} = \varphi_1(\mathcal{X}_v^1)$. For this, the inner product structure is given by

$$\begin{aligned}
&\langle c = 0, i = 0, 0^{\otimes(\mathfrak{m}-3)}|\, U_{\mathcal{G}}\,|v\rangle\,|0\rangle^{\otimes \mathfrak{m}} \qquad (27)\\
&= \frac{1}{2\sqrt{2}sk_v^0} \sum_{l,l'} x_{r(v,l)}\, \delta_{r(v,l),r(v,l')}\\
&= \frac{1}{2\sqrt{2}sk_v^c} \sum_{l} x_{r(v,l)}\\
&= \frac{1}{2\sqrt{2}s} \sum_{u \in \mathcal{N}_v^1} \frac{x_u}{|\mathcal{N}_v^1|} \qquad (28)\\
&= \frac{1}{2\sqrt{2}s} \varphi(\mathcal{X}_v^1),
\end{aligned}$$

with $U_{\mathcal{G}}$ the QMME operator, k_v^c the the number of c-hop neighbors of v, and s the adjacency matrix sparsity. The scalar x_j denotes the feature value indexed by j, while the function $r(v,l)$ returns the ℓ-th neighbor of node v. The Kronecker delta $\delta_{r(v,l),r(v,l')}$ equals 1 if the two neighbors coincide and 0 otherwise.

This expression naturally generalizes to arbitrary indices v, c, i, yielding

$$\langle c, i|\,\langle 0|^{\otimes 5+2\mathfrak{s}}\, U_{\mathcal{G}}\,|v\rangle\,|0\rangle^{\otimes \mathfrak{m}} = \frac{\varphi_i(\mathcal{X}_v^i)}{2\sqrt{2}s^c}. \tag{29}$$

To characterize the state structure more explicitly, consider the computational basis $\{|i\rangle\}_{i=0}^{7}$ of the main register. Satisfies

$$\langle 0^{\otimes(\mathfrak{m}-3)}|\,\perp\rangle = 0 \quad , \tag{30}$$

which guarantees a clean separation between the *moment* component (associated with the ancilla zero state) and the *orthogonal* component (supported on the orthogonal subspace).

Step-by-step derivation of the application of $U_{\mathcal{G}}$, the QMME unitary, on an input state is described in Figure 4.

IV. QUANTUM CLASSIFICATION FRAMEWORK

The QMME protocol will be applied to both the register encoding the superposition of the labeled nodes indexes, and to the register encoding the test node index. This quantum node embedding method maps a node in a feature-labeled network into its moments, which are amplitude-encoded in a quantum state.

The swap-test classifier computes the overlap between two quantum states. This computation yields $|\langle\psi|\phi\rangle|^2$ for two pure states $|\psi\rangle$ and $|\phi\rangle$, and more generally gives

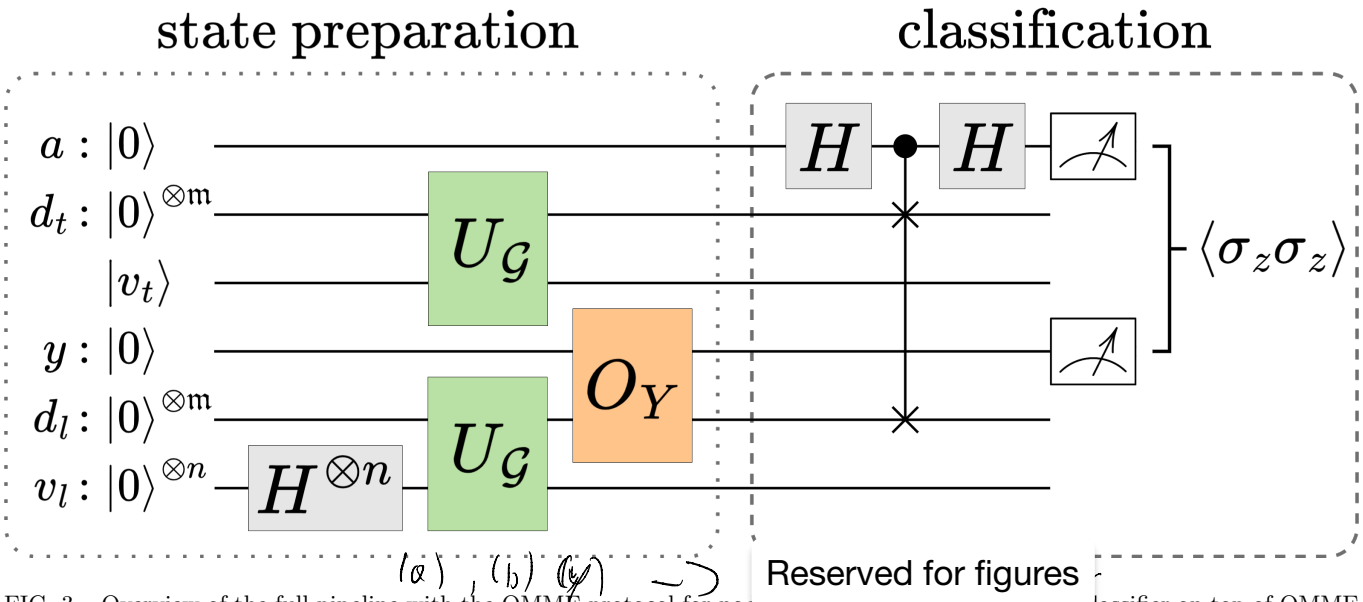


FIG. 3. Overview of the full pipeline with the QMME protocol for node classification. The swap-test classifier on top of QMME state preparation. The first register corresponds to the ancilla qubit (a), the second the test data qubits (d_t), the third the index qubits of the test node (v_t), the fourth stores the label qubit (y), the fifth the labeled data qubits (d_l), and the sixth the index qubits of the labeled nodes (v_l). The operator $U_{\mathcal{G}}$, together with the Hadamard gate and the O_Y oracle, are applied to prepare the input quantum state necessary for the classification procedure. The state is then processed by a quantum classifier $\mathcal{C}$ to predict node labels $\hat{y}_v \in \{0, 1\}$: The classification outcome is extracted via a swap-test and two-qubit measurement statistics on the ancilla and label registers. Measurements are repeated for each unlabeled node, either sequentially or in parallel.

Reserved for figures

circuit computers are universal. There must be a circuit to do this. Maybe we can say, more involved? not not so obvious?

$\mathrm{Tr}(\rho\sigma)$ for two (poss
tum optics, the swap-test has a simple physical implementation, not so obvious for gate-based quantum computers, such as superconducting ones from IBM and Google, and the IonQ's trapped-ion quantum computer [34].

Our quantum classification is illustrated in fig. 3. The first step, state preparation, includes applications of the U_G embedding unitary for both test and labeled data. The final state is an application of the swap-test classifier [25].

The full pipeline of fig. 3 shows the quantum circuit which evolves the state as

$$|0\rangle |0\rangle^{\otimes(\mathrm{m})} |v_t\rangle |0\rangle |0\rangle^{\otimes \mathrm{m}} |0\rangle^{\otimes n} \tag{31}$$

$$\mapsto \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |0\rangle^{\otimes \mathrm{m}} |v_t\rangle |0\rangle |0\rangle^{\otimes \mathrm{m}} |v_l\rangle \tag{32}$$

$$\mapsto \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |\psi_t\rangle |v_t\rangle |0\rangle |\psi_l\rangle |v_l\rangle \tag{33}$$

$$\mapsto \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |\psi_t\rangle |v_t\rangle |y_l\rangle |\psi_l\rangle |v_l\rangle \tag{34}$$

$$\mapsto \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |\psi_t\rangle |\psi_l\rangle |v_t\rangle |y_l\rangle |v_l\rangle \tag{35}$$

$$\mapsto \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} (|0\rangle |\psi_+\rangle + |0\rangle |\psi_-\rangle) |v_t\rangle |y_l\rangle |v_l\rangle \tag{36}$$

$$\tag{37}$$

where $|\psi_\pm\rangle = |\psi_t\rangle |\psi_l\rangle \pm |\psi_l\rangle |\psi_t\rangle$.

Classical simulations of quantum circuits. [ongoing] performance against state-of-the-art classical baselines.

A. Dataset Description

B. Results

Finally, to validate our code, we replicated the simplest swap-test classifier example from the reference paper, involving two nodes each with two features. This test confirmed the correctness of our swap-test classifier implementation and helped isolate errors unrelated to core classification logic.

VI. COMPLEXITY ANALYSIS

We analyze the complexity of the QMME circuit in terms of gate count, circuit depth, and required qubits. Let $N = 2^m$ be the number of labeled nodes and d' the precision (in bits) of the encoded features and degrees.

Loading a feature vector via $\tilde{O}_X$ maps $|v\rangle |0\rangle \mapsto |v\rangle |\tilde{x}_v\rangle$ and can be implemented with $\mathcal{O}$(did not find) gates using structured memory access or qRAM. The rotation

Don't we know this?

F_X applies a controlled $R_y(2\arccos x_v)$ gate, with θ computed on-the-fly via quantum arithmetic, contributing $\mathcal{O}(\text{poly } n + d')$ gates. The oracle $\tilde{O}_{K^{-1}}$ and $F_{K^{-1}}$ follow the same complexity.

The total number of oracle calls per QMME state preparation is constant, with two calls to each oracle (and its inverse), plus up to five amplitude-encoding operations. The number of Hadamard gates scales as $\mathcal{O}(n + \mathfrak{s})$, and the total gate count is dominated by arithmetic and oracle logic.

Work qubits include l for input registers, $\mathfrak{s}$ for neighbors basis-encoding, and $\mathcal{O}(1)$ for control. The total number of qubits is thus $\mathcal{O}(n+l+\mathfrak{s})$, where d' depends on precision.

Metric	Complexity Estimate
Gate count	$\mathcal{O}(\text{poly}(n) + l)$
Circuit depth	$\mathcal{O}(\text{poly}(n) + l)$
Work qubits	$\mathcal{O}(d' + \mathfrak{s})$
Total qubits	$\mathcal{O}(n + d' + \mathfrak{s})$

[be very clear on constraints of current quantum hardware][Get total cost per run (both as in QMME and in

This needs work. I think we need a full discussion on the sparcity of the network or not. We sould have two columns. Sparce and non-sparse. We should make it as a function of N and not n l think.

on synthetic data. Future work includes extensions to deeper moments, weighted graphs, and implementation on near-term hardware.

An architecture that others can build on (generalizable moment-encoding circuit) [22]. Nevertheless, the QMME protocol may be applied to any node-level binary classification on graph-structured data with feature-labeled nodes.

Potential improvements or extensions include multiclass classification (either gene categories or even outside the scope of biological networks), and mention applications to other types of nets.

[mention limitations] [adress o porquê de escolher momentos até ordem 4. e de como cada parte do circuito (1-hop, 2-hop) contribui para a performance]

[1] H. Horn, M. S. Lawrence, C. R. Chouinard, Y. Shrestha, J. X. Hu, E. Worstell, E. Shea, N. Ilic, E. Kim, A. Kamburov, *et al.*, Nature methods **15**, 61 (2018).

[2] M. Nourbakhsh, K. Degn, A. Saksager, M. Tiberti, and E. Papaleo, Briefings in Bioinformatics **25**, bbad519 (2024).

[3] A. C. Gumpinger, K. Lage, H. Horn, and K. Borgwardt, Bioinformatics **36**, i508 (2020).

[4] L. A. Garraway and E. S. Lander, Cell **153**, 17 (2013).

[5] B. Vogelstein, N. Papadopoulos, V. E. Velculescu, S. Zhou, L. A. Diaz Jr, and K. W. Kinzler, science **339**, 1546 (2013).

[6] G. M. Frampton, A. Fichtenholtz, G. A. Otto, K. Wang, S. R. Downing, J. He, M. Schnall-Levin, J. White, E. M. Sanford, P. An, *et al.*, Nature biotechnology **31**, 1023 (2013).

[7] M. A. Reyna, M. D. Leiserson, and B. J. Raphael, Bioinformatics **34**, i972 (2018).

[8] I. A. Kovács, K. Luck, K. Spirohn, Y. Wang, C. Pollis, S. Schlabach, W. Bian, D.-K. Kim, N. Kishore, T. Hao, *et al.*, Nature communications **10**, 1240 (2019).

[9] F. Cheng, I. A. Kovács, and A.-L. Barabási, Nature communications **10**, 1197 (2019).

[10] K. Lage, E. O. Karlberg, Z. M. Størling, P. I. Olason, A. G. Pedersen, O. Rigina, A. M. Hinsby, Z. Tümer, F. Pociot, N. Tommerup, *et al.*, Nature biotechnology **25**, 309 (2007).

[11] T. Li, R. Wernersson, R. B. Hansen, H. Horn, J. Mercer, G. Slodkowicz, C. T. Workman, O. Rigina, K. Rapacki, H. H. Stærfeldt, *et al.*, Nature methods **14**, 61 (2017).

[12] D. Szklarczyk, A. L. Gable, D. Lyon, A. Junge, S. Wyder, J. Huerta-Cepas, M. Simonovic, N. T. Doncheva, J. H. Morris, P. Bork, *et al.*, Nucleic acids research **47**, D607 (2019).

[13] J. G. Lohr, P. Stojanov, M. S. Lawrence, D. Auclair, B. Chapuy, C. Sougnez, P. Cruz-Gordillo, B. Knoechel, Y. W. Asmann, S. L. Slager, *et al.*, Proceedings of the National Academy of Sciences **109**, 3879 (2012).

[14] M. S. Lawrence, P. Stojanov, C. H. Mermel, J. T. Robinson, L. A. Garraway, T. R. Golub, M. Meyerson, S. B. Gabriel, E. S. Lander, and G. Getz, Nature **505**, 495 (2014).

[15] P. Wittek, *Quantum machine learning: what quantum computing means to data mining* (Academic Press, 2014).

[16] M. Schuld, I. Sinayskiy, and F. Petruccione, Contemporary Physics **56**, 172 (2015).

[17] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Nature **549**, 195 (2017).

[18] D. K. Park, C. Blank, and F. Petruccione, Physics Letters A **384**, 126422 (2020).

[19] J. P. Moutinho, A. Melo, B. Coutinho, I. A. Kovács, and Y. Omar, Physical Review A **107**, 032605 (2023).

[20] D. Magano, J. Moutinho, and B. Coutinho, SciPost Physics Core **6**, 058 (2023).

[21] J. P. Moutinho, D. Magano, and B. Coutinho, Scientific Reports **14**, 1026 (2024).

[22] W. Bi, L. Du, Q. Fu, Y. Wang, S. Han, and D. Zhang, in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining* (2023) pp. 132–140.

[23] H. Zhao, L. Lewis, I. Kannan, Y. Quek, H.-Y. Huang, and M. C. Caro, PRX Quantum **5**, 040306 (2024).

[24] Y. Wang and J. Liu, Reports on Progress in Physics (2024).

[25] C. Blank, D. K. Park, J.-K. K. Rhee, and F. Petruccione, npj Quantum Information **6**, 41 (2020).

[26] J. Shao, *Mathematical statistics* (Springer, 1999).

[27] O. Goldreich and D. Ron, in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing* (1997) pp. 406–415.

[28] O. Goldreich, *Introduction to property testing* (Cambridge University Press, 2017).

[29] S. Ben-David, A. M. Childs, A. Gilyén, W. Kretschmer, S. Podder, and D. Wang, SIAM Journal on Computing

$$
\begin{aligned}
|v,0^{\otimes}\rangle &\xrightarrow{I_{n+3}\otimes H^{\otimes \mathfrak{s}}\otimes I_{\mathfrak{s}+5}} \frac{1}{\sqrt{s}}|v,0^{\otimes 3}\rangle \sum_{\ell_1\in[s]} |\ell_1,|0^{\otimes \mathfrak{s}+5}\rangle\rangle \\
&\xrightarrow{O_L} \frac{1}{\sqrt{s}}|v,0^{\otimes 3}\rangle \sum_{\ell_1\in[s]} |r(v,\ell_1),|0^{\otimes \mathfrak{s}+5}\rangle\rangle \\
&\xrightarrow{I_n\otimes H\otimes I_{2+\mathfrak{s}+5}} \frac{1}{\sqrt{2s}}|v\rangle \sum_{c\in[2]}\sum_{\ell_1\in[s]} |c,0^{\otimes 2},r(v,\ell_1),|0^{\otimes \mathfrak{s}+5}\rangle\rangle \\
&\xrightarrow{I_{n+3+\mathfrak{s}}C-H^{\otimes \mathfrak{s}}\otimes I_5} \frac{1}{\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]} \left(|0_c,0^{\otimes 2},r(v,\ell_1),|0^{\otimes \mathfrak{s}}\rangle\rangle + \frac{1}{\sqrt{s}}\sum_{\ell_2\in[s]} |1_c,0^{\otimes 2},r(v,\ell_1),\ell_2\rangle\right)|0^{\otimes 5}\rangle \\
&\xrightarrow{C-O_L} \frac{1}{\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]} \left(|0_c,0^{\otimes 2},r(v,\ell_1),|0^{\otimes \mathfrak{s}}\rangle\rangle + \frac{1}{\sqrt{s}}\sum_{\ell_2\in[s]} |1_c,0^{\otimes 2},r(v,\ell_1),r(r(v,\ell_1),\ell_2)\rangle\right)|0^{\otimes 5}\rangle \\
&\xrightarrow{I_{n+1}\otimes H^{\otimes 2}\otimes I_{2\mathfrak{s}+5}} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]}\sum_{i\in[4]} \left(|0_c,i,r(v,\ell_1),|0^{\otimes \mathfrak{s}}\rangle\rangle + \frac{1}{\sqrt{s}}\sum_{\ell_2\in[s]} |1_c,i,r(v,\ell_1),r(r(v,\ell_1),\ell_2)\rangle\right)|0^{\otimes 5}\rangle \\
&\xrightarrow{C-O_X} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]}\sum_{i\in[4]} \left(x^i_{r(v,\ell_1)}|0_c,i,r(v,\ell_1),|0^{\otimes \mathfrak{s}}\rangle\rangle + \frac{1}{\sqrt{s}}\sum_{\ell_2\in[s]} x^i_{r(r(v,\ell_1),\ell_2)}|1_c,i,r(v,\ell_1),r(r(v,\ell_1),\ell_2)\rangle\right)|0^{\otimes 5}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{C-O_L} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]}\sum_{i\in[4]} \left(x^i_{r(v,\ell_1)}|0_c,i,r(v,\ell_1),|0^{\otimes \mathfrak{s}}\rangle\rangle + \frac{1}{\sqrt{s}}\sum_{\ell_2\in[s]} x^i_{r(r(v,\ell_1),\ell_2)}|1_c,i,r(v,\ell_1),\ell_2\rangle\right)|0^{\otimes 5}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{I_{n+3+\mathfrak{s}}C-H^{\otimes \mathfrak{s}}\otimes I_5} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]}\sum_{i\in[4]} \left(x^i_{r(v,\ell_1)}|0_c,i,r(v,\ell_1)\rangle + \frac{1}{\sqrt{s}}\sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i|1_c,i,r(v,\ell_1)\rangle\right)|0^{\otimes \mathfrak{s}+5}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{O_L} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{\ell_1\in[s]}\sum_{i\in[4]} \left(x^i_{r(v,\ell_1)}|0_c,i,\ell_1\rangle + \frac{1}{s}\sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i|1_c,i,\ell_1\rangle\right)|0^{\otimes \mathfrak{s}+5}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{I_{n+3}H^{\otimes \mathfrak{s}}\otimes I_{\mathfrak{s}+5}} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{i\in[4]} \left(\sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i|0_c\rangle + \frac{1}{s}\sum_{x''\in\mathcal{X}^2_v} x''^i|1_c\rangle\right)|i,|0^{\otimes 2\mathfrak{s}+5}\rangle\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{O_{K^{-1}}} \frac{1}{2\sqrt{2s}}|v\rangle \sum_{i\in[4]} \left(\sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i|0_c\rangle + \frac{1}{s}\sum_{x''\in\mathcal{X}^2_v} x''^i|1_c\rangle\right)\frac{1}{k^c_v}|i,|0^{\otimes 2\mathfrak{s}+5}\rangle\rangle + |v\rangle|\perp\rangle \\
&= |v\rangle\left(\sum_{i,c}(2\sqrt{2}s^{c+1})^{-1}\sum_{x'''\in\mathcal{X}^c_v}\frac{x'''^i}{k^c_v}|c,i,0^{\otimes(2\mathfrak{s}+5)}\rangle + |\perp\rangle\right) \\
&= |v\rangle\left(\sum_{i,c}(2\sqrt{2}s^{c+1})^{-1}\varphi(\mathcal{X}^c_v)|c,i,0^{\otimes(2\mathfrak{s}+5)}\rangle + |\perp\rangle\right) \\
&= |v\rangle\left(\sum_{j=0}^{j=7} c_j m_{v,j}|j,0^{\otimes(2\mathfrak{s}+5)}\rangle + |\perp\rangle\right)
\end{aligned}
$$

FIG. 4. Step-by-step derivation of the application of $U_{\mathcal{G}}$, the QMME unitary, on an input state. Quantum circuit transformation steps for computing node-neighborhood moment embeddings.

53, FOCS20 (2024).

[30] L. Lin, arXiv preprint arXiv:2201.08309 (2022).

[31] D. Camps, L. Lin, R. Van Beeumen, and C. Yang, SIAM Journal on Matrix Analysis and Applications **45**, 801 (2024).

[32] P. Kuklinski and B. Rempfer, arXiv preprint arXiv:2401.04234 (2024).

[33] Y. Tong, D. An, N. Wiebe, and L. Lin, Physical Review A **104**, 032422 (2021).

[34] L. Cincio, Y. Subaşı, A. T. Sornborger, and P. J. Coles, New Journal of Physics **20**, 113022 (2018).

[35] M. Möttönen, J. J. Vartiainen, V. Bergholm, and M. M.

Salomaa, Physical review letters **93**, 130502 (2004).
[36] S. Arora and B. Barak, *Computational complexity: a modern approach* (Cambridge University Press, 2009).
[37] R. Levi, R. Rubinfeld, and A. Yodpinyanee, in *Proceedings of the 27th ACM symposium on Parallelism in Algorithms and Architectures* (2015) pp. 59–61.
[38] J. Biamonte, M. Faccin, and M. De Domenico, Communications Physics **2**, 53 (2019).
[39] J. M. Martyn, Z. M. Rossi, A. K. Tan, and I. L. Chuang, PRX quantum **2**, 040203 (2021).
[40] S. Behnezhad, M. Roghani, and A. Rubinstein, in *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)* (IEEE, 2023) pp. 2322–2335.